



هوش مصنوعی

بهار ۱۴۰۵

استاد درس: دکتر احسان تن قطاری

مهلت ارسال: ۵ مرداد

پروژه درس هوش مصنوعی

- جزئیات پروژه‌ی نهایی درس در این سند قابل مشاهده است. نمره‌ی کل پروژه از ۴ نمره می‌باشد که در قالب گروه‌های ۲ نفری ارائه می‌شود.
- ابتدا یک فرم برای مشخص کردن گروه‌ها در کوئرا در اختیاران قرار داده می‌شود. مهلت پر کردن این فرم تا ساعت ۱۲ ظهر روز جمعه ۵ تیر می‌باشد. از هر گروه تنها یک نفر باید فرم را پر کند و همچنین دانشجویانی که فرم را پر نکرده باشند، به صورت تصادفی با یکدیگر همگروهی خواهند شد.
- بعد از مشخص شدن گروه‌ها، فرم انتخاب پروژه در ساعت ۸ شب روز جمعه ۵ تیر در کوئرا قرار داده خواهد شد و مهلت پر کردن آن تا ۱۲ ظهر فردای آن روز می‌باشد. گروه‌ها بر اساس زمان پر کردن این فرم برای تخصیص پروژه‌ها اولویت‌بندی می‌شوند. ظرفیت هر پروژه ۶ تیم می‌باشد.
- پروژه‌های ۱ و ۳ شامل ۱۰ درصد نمره‌ی امتیازی هستند که روی کل درس سرریز نمی‌شوند. پروژه‌های ۲ و ۴ شامل بخش امتیازی رقابتی هستند که برای تیم اول ۰.۴ نمره، تیم دوم ۰.۲ نمره و تیم سوم ۰.۱ نمره امتیازی روی کل درس اعمال می‌شود.
- ددلاین آپلود خروجی نهایی و گزارش پروژه تا ساعت ۲۳:۵۹:۵۹ روز دوشنبه ۵ مرداد می‌باشد. پس از آن اطلاعیه‌ای برای تحویل مجازی پروژه‌ها اعلام می‌شود.
- سوالات خود را راجع به روند پروژه در پیام مربوطه داخل کوئرا مطرح کنید.

صورت پروژه‌ها

پروژه ۱. شبکه‌های عصبی بازگشتی برای تشخیص فعالیت انسانی (۱۱۰ نمره)

۱. مقدمه و هدف

تصور کنید فقط از روی تکان‌های گوشی داخل جیب بتوان فهمید صاحبش در حال راه رفتن است، دویدن یا نشسته؛ در این پروژه دقیقاً همین سیستم را می‌سازید. هر گوشی یک شتاب‌سنج دارد که چندین بار در ثانیه شتاب بدن را در سه محور x ، y و z ثبت می‌کند، و این مقادیر پشت سر هم یک دنباله‌ی زمانی می‌سازند که الگوی حرکتی فرد در آن نهفته است. راه رفتن یک موج منظم، دویدن همان موج با دامنه‌ی بیشتر، و نشستن تقریباً یک خط صاف. مسئله‌ی تشخیص فعالیت انسانی (Human Activity Recognition) همین است که از روی این دنباله حدس بزنیم فرد چه می‌کند. چون ترتیب زمانی داده‌ها معنا می‌سازد و مدل باید نوعی حافظه داشته باشد، این مسئله تمرینی ایده‌آل برای شبکه‌های بازگشتی است.

شبکه‌های بازگشتی در درس پوشش داده نشده‌اند، پس پیش از شروع مرجع [۱] را مطالعه کنید که مقدمه‌ای روشن بر این شبکه‌هاست و پایه‌ی نظری لازم برای کل پروژه را فراهم می‌کند. هدف آن است که نخست یک شبکه‌ی بازگشتی ساده را از پایه و فقط با NumPy بسازید تا سازوکارش را واقعاً بفهمید، و سپس با معماری‌های آماده‌ی PyTorch مدل قوی‌تری برای همین مسئله بسازید و مقایسه کنید. در بخش اول استفاده از ماژول‌هایی مانند nn.RNN مجاز نیست؛ ساختن مدل از صفر فهم شما را از آنچه پشت ابزارهای آماده می‌گذرد بسیار عمیق‌تر می‌کند.

۱. مجموعه داده و آماده‌سازی

از مجموعه داده‌ی متن‌باز WISDM [۳] استفاده می‌کنید که ثبت شتاب‌سنج گوشی از چند کاربر هنگام شش فعالیت است: راه رفتن، دویدن، بالا و پایین رفتن از پله، نشستن و ایستادن. هر نمونه شامل شناسه‌ی کاربر، برجسب فعالیت و سه مؤلفه‌ی شتاب (a_x, a_y, a_z) است.

داده‌ی خام را نخست پنجره‌بندی کنید، یعنی دنباله‌ی هر کاربر را به قطعه‌های هم‌پوشان با طول ثابت T ببرید و برجسب هر پنجره را فعالیت غالب آن بگیرید؛ سپس داده را استاندارد کنید و میانگین μ و انحراف معیار σ را فقط از داده‌ی آموزش حساب کنید تا چیزی از داده‌ی آزمون نشت نکند: $x_{\text{norm}} = (x - \mu) / \sigma$. تقسیم آموزش و آزمون باید بر اساس کاربر باشد، یعنی هیچ کاربری هم‌زمان در هر دو نباشد؛ در غیر این صورت پنجره‌های مشابه در هر دو می‌افتند و دقت به‌شکلی فریبنده بالا می‌رود. چون توزیع کلاس‌ها نامتوازن است، در کنار دقت باید macro-F1 و ماتریس درهم‌ریختگی (Confusion Matrix) را نیز گزارش کنید.

۲. بخش اول: شبکه‌ی بازگشتی ساده از صفر

یک شبکه‌ی بازگشتی ساده با یک لایه‌ی پنهان را فقط با NumPy بسازید که طبقه‌بندی را بر پایه‌ی حالت آخرین گام h_T انجام می‌دهد:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad \hat{y} = \text{softmax}(W_{hy}h_T + b_y).$$

سپس پس‌انتشار در زمان (Back-Propagation Through Time) را پیاده کنید؛ چون W_{hh} میان همه‌ی گام‌ها مشترک است، گرادیانش جمع سهم همه‌ی گام‌هاست:

$$\delta_t = (W_{hh}^\top \delta_{t+1}) \odot (1 - h_t^2), \quad \frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^T \delta_t h_{t-1}^\top.$$

برای لایه‌ی خروجی از softmax و خطای Cross-Entropy استفاده کنید (می‌توانید پیاده‌سازی آماده‌ی آن‌ها را به کار ببرید) و برای پایداری از بریدن گرادیان (Gradient Clipping) بهره ببرید. پیش از آموزش، درستی پیاده‌سازی را با بررسی عددی گرادیان تأیید کنید: گرادیان تحلیلی را با تقریب تفاضل مرکزی $(\mathcal{L}(\theta + \varepsilon) - \mathcal{L}(\theta - \varepsilon)) / 2\varepsilon$ مقایسه کنید و خطای نسبی کمتر از 10^{-5} بگیرید. در گزارش هم منحنی کاهش خطای آموزش و هم نتیجه‌ی بررسی گرادیان را بیاورید.

۳. بخش دوم: معماری‌های آماده با PyTorch

ابتدا به‌عنوان بررسی سلامت نشان دهید شبکه‌ی NumPy شما و nn.RNN با وزن‌های یکسان خروجی تقریباً برابر می‌دهند. سپس دو مدل را آموزش و مقایسه کنید: همان شبکه‌ی ساده و LSTM. شبکه‌ی ساده روی دنباله‌های بلند به‌خاطر محو شدن گرادیان ضعیف است؛ LSTM [۲] این مشکل را با یک حالت سلولی و چند دروازه حل می‌کند:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t).$$

در گزارش توضیح دهید چرا این ساختار از محو شدن گرادیان جلوگیری می‌کند. در پایان یک LSTM دوسویه (Bidirectional) هم آموزش دهید و ببینید آیا دیدن ادامه‌ی پنجره به نتیجه کمک می‌کند.

۴. بخش سوم: تحلیل

نرم گرادیان را برحسب فاصله‌ی زمانی برای شبکه‌ی ساده و LSTM رسم کنید و تفاوتشان را در حفظ وابستگی‌های دور توضیح دهید. سپس با دو آزمایش کوتاه اثر طول پنجره T و اندازه‌ی لایه‌ی پنهان را بر دقت بسنجید و نتیجه را تفسیر کنید؛ برای مثال توضیح دهید چرا پنجره‌ی بسیار کوتاه یا بسیار بلند می‌تواند دقت را کم کند. همچنین F1 هر کلاس را جداگانه گزارش کنید تا مشخص شود مدل روی کدام فعالیت‌ها ضعیف‌تر است.

۵. تحویلی‌ها

خروجی را در پوشه‌ای به نام شماره‌ی دانشجویی خود فشرده (zip) کنید: یک نوت‌بوک ژوپیتِر (ipynb). شامل بخش اول و دوم با خروجی‌های اجراشده، و یک گزارش کوتاه PDF که روش کار، جدول‌ها، نمودارها و پاسخ پرسش‌های تحلیلی را در بر بگیرد. ضمیمه کردن وزن‌های مدل اختیاری است، اما مطمئن شوید نوت‌بوک به‌ترتیب و بدون خطا اجرا می‌شود.

۶. معیارهای ارزیابی

نتایج همه‌ی مدل‌ها را روی مجموعه‌ی آزمون تقسیم‌شده بر اساس کاربر در جدول زیر گرد آورید:

مدل	دقت	macro-F1	زمان آموزش
شبکه‌ی ساده (از صفر)			
(PyTorch) LSTM			
Bidirectional LSTM			

ماتریس درهم‌ریختگی بهترین مدل را رسم کنید و بگویید کدام دو فعالیت بیشتر با هم اشتباه می‌شوند. در گزارش به دو پرسش پاسخ دهید: اختلاف دقت میان تقسیم تصادفی و تقسیم بر اساس کاربر چقدر است و کدام صادقانه‌تر است؟ و چرا LSTM از شبکه‌ی ساده بهتر عمل می‌کند و این به محو شدن گرادیان چه ربطی دارد؟ تقسیم‌بندی نمره پروژه نیز به شکل زیر می‌باشد:

بخش	درصد نمره
بخش اول: شبکه‌ی بازگشتی از صفر و بررسی عددی گرادیان	۳۵٪
بخش دوم: معماری‌های آماده با PyTorch و مقایسه	۳۰٪
بخش سوم: تحلیل	۱۵٪
گزارش و کیفیت ارائه	۲۰٪
بخش امتیازی (اختیاری)	تا ۱۰٪

۷. بخش امتیازی (تا ۱۰٪)

با انجام یکی از دو مورد زیر تا ۱۰ درصد نمره‌ی اضافه می‌گیرید. نخست افزودن و آموزش معماری GRU (نسخه‌ی ساده‌تر LSTM با دو دروازه) و مقایسه‌ی آن با LSTM از نظر دقت و سرعت آموزش. گزینه‌ی دوم ارزیابی به روش Leave-One-Subject-Out روی زیرمجموعه‌ای از کاربران است که برآورد صادقانه‌تری از تعمیم مدل می‌دهد.

اگر با مفاهیمی مانند پس‌انتشار در زمان (BPTT)، محو شدن گرادیان، نقش دروازه‌های LSTM، نشت داده در تقسیم مبتنی بر کاربر، تفاوت macro-F1 و micro-F1، یا softmax و Cross-Entropy آشنا نیستید، مطالعه‌ی کوتاه آن‌ها پیش از شروع به شما کمک می‌کند.

مراجع

- [۱] Z. C. Lipton, J. Berkowitz, and C. Elkan, “A Critical Review of Recurrent Neural Networks for Sequence Learning,” arXiv:1506.00019, 2015.
- [۲] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, 1997.
- [۳] J. R. Kwapisz, G. M. Weiss, and S. A. Moore, “Activity Recognition using Cell Phone Accelerometers,” ACM SIGKDD Explorations Newsletter, vol. 12, no. 2, 2011.